

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 649 — Physically-Based Modeling

Particles, rigid bodies, deformables, constraints, and fluids — simulated well enough to ship

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 3 (with CSCE 647 and CSCE 608)

Prerequisites: CSCE 641 Computer Graphics; calculus and linear algebra; ordinary differential equations helpful but developed here as needed

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A simulation library: particles, constraints, collision detection and response, rigid bodies, cloth, and a fluid solver — written by you, with energy and momentum behaviour measured rather than assumed

Course description

CSCE 647 computed where light goes. This course computes where *everything else* goes. The subject is the numerical simulation of physical systems at the fidelity graphics needs, which is a specific and interesting standard: **plausible, stable, controllable, and fast** — in that order, and explicitly not 'accurate'.

That ordering is the thing to understand early. An engineering simulation that is 2% wrong may be useless. A film simulation that is 2% wrong is fine, and one that is perfectly accurate but explodes on frame 180, or cannot be art-directed, is useless. Several of the most important techniques in this course are *provably less accurate* than the alternatives and are used anyway, for reasons the course will make precise.

Everything here reduces to one structure: a state vector, a function that gives its derivative, and a scheme for advancing it in time. Particles, cloth, rigid bodies, and fluids differ in what is in the state vector and how the derivative is computed, and almost nothing else. **Once you see that, the field stops being a collection of unrelated tricks.**

You will write all of it. By the end you will have a simulator you can reason about when it misbehaves — which it will, because every simulator does, and knowing whether you are looking at a stability problem, a constraint problem, or a collision problem is the skill this course exists to build.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Express any physical system as a state vector and a derivative function.
- 2 Choose an integrator from the stability and energy behaviour a system requires.
- 3 Explain stiffness and why it forces implicit integration.
- 4 Implement constraints by projection and by Lagrange multipliers, and say when each is appropriate.
- 5 Build a collision pipeline with correct broad and narrow phases.
- 6 Compute collision response with restitution and Coulomb friction, including resting contact.
- 7 Simulate rigid bodies with correct rotational dynamics.
- 8 Implement a deformable solid and explain why mass-spring models fail.
- 9 Implement both a grid-based and a particle-based fluid solver.
- 10 Diagnose an unstable simulation and name the mechanism.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Simulation as an Initial Value Problem	What is a simulator actually doing?
2	Numerical Integration	How do you advance a state vector in time without it blowing up?
3	Particle Systems and Springs	What can you build out of masses and springs?
4	Constraints	How do you make something exactly inextensible?
5	Collision Detection	How do you find all contacts among ten thousand moving objects?
6	Collision Response	How do you make a stack of boxes stand still?
7	Rigid Body Dynamics	Why does a tumbling object flip over on its own?
8	Articulated Bodies	How do you simulate a skeleton that does not come apart?
9	Deformable Solids	How do you simulate a material rather than a mesh?
10	Cloth and Hair	Why is a bedsheet harder to simulate than a boulder?
11	Fluids I: Grid-Based	How do you simulate smoke and water on a grid?
12	Fluids II: Particles and Hybrids	What do particles fix, and what do they break?
13	Shipping a Simulator	What stands between a working simulation and a usable one?

Projects

Project 1 — Particles, constraints, and contact (after Module 7)

The core simulator. Rigid bodies and everything after build on this machinery, so the integrator, the constraint solver, and the contact handling all have to be right rather than merely stable-looking.

Requirements

- A generic integrator interface with explicit Euler, symplectic Euler, velocity Verlet, and RK4 behind it, selectable at runtime.
- A particle system with gravity, drag, and damped springs.
- A mass-spring cloth with structural, shear, and bending springs.
- Broad-phase collision detection with a spatial hash or sweep and prune, with the pair count reported.
- Impulse-based collision response with restitution and Coulomb friction.
- Rigid body dynamics with a correct inertia tensor and quaternion orientation.

Done means

- **An energy plot for an undamped orbiting particle under each integrator, over 10,000 steps.** Explicit Euler must gain energy, symplectic Euler must oscillate around a constant, RK4 must slowly lose it. Explain each curve.
- A cloth dropped onto a sphere, draping and coming to rest without jitter, with the resting energy reported.
- **A stack of ten boxes that remains standing for sixty seconds.** This is harder than it sounds and is the real test of the contact solver.
- **The Dzhanibekov effect reproduced:** a freely tumbling rigid body with three distinct principal moments flipping periodically. Angular momentum must be conserved to floating-point tolerance — report the drift.

Project 2 — Deformables or fluids (after Module 12)

Take one of the two hard systems to a standard you would be willing to put in a reel, and measure it rather than trusting it.

Requirements

- **Either:** a corotational FEM deformable solid with collision handling and an implicit solver; **or** a fluid solver — Eulerian with pressure projection, or SPH/PBF, or FLIP.
- An implicit or semi-implicit time integration scheme, with the linear solve explained.
- Self-collision handling, or free-surface handling for a fluid.
- A demonstration that the solver converges, with iteration count against residual plotted.
- Parameters exposed in physically meaningful units.

Done means

- A video of the system under at least three qualitatively different conditions.
- **A volume-loss plot for a fluid, or an energy plot for a deformable**, with an honest account of what is lost and where.
- A stability study: the largest time step at which the system survives, for each integration scheme you implemented.
- **A documented failure.** Find the configuration that breaks your simulator, show it, and explain the mechanism. Every simulator has one; not knowing yours is the problem.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Baraff & Witkin — Physically Based Modeling (SIGGRAPH course notes, free)	The primary text. Differential equation basics, particle systems, constraints, rigid bodies, and collision — written with unusual clarity and still the best free introduction to the field.
Matthias Müller — Ten Minute Physics (free video and code)	Short, complete, working implementations of almost every technique in this course, with the derivations. Watch the relevant episode before each module.
Bridson — Fluid Simulation for Computer Graphics (course notes, free)	The standard reference for Modules 11 and 12. The full book is worth owning; the free course notes cover the essentials.
Erin Catto — GDC talks and Box2D (free)	How constraint solvers are built in practice, from the author of Box2D. The sequential impulse material is the clearest explanation of Module 06 that exists.

Resource	Role
Bargteil & Shinar — An Introduction to Physics-Based Animation (SIGGRAPH course, free)	A modern survey tying the whole field together. Good second voice throughout, and strong on deformables.
Müller et al. — Position Based Dynamics papers (free)	PBD and XPBD, from their authors. The primary sources for Module 04.

Tooling

- **C++17 with Eigen.** You will be solving sparse linear systems from Module 10 onward, and the performance matters once particle counts rise.
- **No physics engine.** Integration, constraints, collision, and solvers are the course. You may use a linear algebra library.
- **A real-time viewer** — Polyscope or a minimal OpenGL harness. You cannot debug a simulation you cannot watch, and stepping frame by frame is essential.
- **Plotting:** Python and matplotlib, or equivalent. **Energy and momentum plots are the primary diagnostic in this course** and you will make hundreds of them.
- **Bullet or Box2D installed as a reference**, not a dependency. When your stack of boxes jitters and theirs does not, the difference is informative.
- **A portfolio repository** with a video for every system, plus the energy plot that shows it is behaving.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.